

Tutoriel complet — MANRS Observatory de zéro

Guide pédagogique : pourquoi chaque décision, flux de données, ordre de développement. Pour quelqu'un qui redémarre le projet ou veut comprendre l'architecture en profondeur.

Date : Juin 2026

Stack : Python/FastAPI/React/PostgreSQL

Objectif : Mesurer la conformité MANRS des 477 opérateurs réseau d'Afrique de l'Ouest

Chapitre 1 — Pourquoi cette architecture ?

Le problème

Tu dois répondre à cette question :

Quel est l'état de la sécurité du routage Internet en Afrique de l'Ouest ?

Pour répondre, tu dois : 1. Collecter des données sur 477 opérateurs réseau (ASN) dans 16 pays 2. Évaluer ces opérateurs sur 4 critères MANRS (filtering, anti-spoofing, coordination, validation) 3. Afficher ça sur une carte interactive 4. Permettre de cliquer sur un pays → voir ses opérateurs → cliquer sur un opérateur → voir sa fiche détaillée

Les données n'existent pas nulle part

Le CDC te dit : "Utilise les APIs MANRS, c'est simple !"

Problème : l'API MANRS v2 ne donne que 2 infos : - Liste des pays + ASN - Statut membre MANRS (oui/non)

Mais elle **ne donne pas** les scores détaillés de chaque action (filtering, anti-spoofing, coordination, validation). Pourquoi ? Parce que MANRS réserve ces données aux opérateurs eux-mêmes.

La solution : agrégation multi-sources

Au lieu de demander à MANRS, tu vas croiser 5 sources publiques pour approximer les 4 actions :

Action	Source	Logique
Filtering	PeeringDB	L'ASN a-t-il un irr_as_set renseigné ? → Si oui, il publie ses politiques de routage
Anti-spoofing	CAIDA Spoofer	L'ASN bloque-t-il le spoofing d'adresses IP ? → Mesure réelle par tests
Coordination	PeeringDB	L'ASN est-il sur PeeringDB ? → Si oui, il maintient des contacts publics
Validation RPKI	RIPE NCC Validator	>50% des préfixes de l'ASN ont-ils un ROA valide ? → Mesure directe

C'est l'**originalité** de ton observatoire : tu es le **premier** à croiser ces 5 sources pour évaluer MANRS sans la coopération des opérateurs.

Flux de données complet

5 SOURCES DE DONNÉES PUBLIQUES

- | | |
|------------------|----------------------------------|
| 1. MANRS v2 | → liste ASN + membres |
| 2. RIPE Stat | → préfixes annoncés |
| 3. RIPE RPKI | → statut ROA (valid/invalid) |
| 4. PeeringDB | → coordination + filtering (IRR) |
| 5. CAIDA Spoofer | → anti-spoofing (% blocking) |

↓

[COLLECTEUR PYTHON]

Interroge les 5 sources, rate limit 0.5s
 Boucle : 16 pays → 477 ASN → 6810 préfixes
 Calcule les 4 actions → Score 0-4

↓

[POSTGRESQL]

4 tables : asn, prefixes, countries, ai_recommendations
 Persistance des données brutes

↓

[API REST FASTAPI]

5 endpoints : /stats, /countries, /countries/{code},
 /asn/{number}, /search

↓

[FRONTEND REACT]

5 pages : Dashboard, Pays, Fiche ASN, Recherche, À propos
 Carte interactive, graphiques, tableaux

Chapitre 2 — Module 1 : La base de données

Pourquoi PostgreSQL ?

- ✓ Relationnelle : tu as des relations (pays → ASN → préfixes)
- ✓ JSON : pour les réponses API, tu retournes du JSON. PostgreSQL gère ça nativement
- ✓ Indexes : tu vas faire des requêtes comme “tous les ASN du Bénin” → faut indexer
- ✓ LATERAL JOIN : tu dois calculer des stats par ASN à la volée (ROA coverage %) → LATERAL JOIN c’est pour ça
- ✓ Gratuit + conteneurisé : Docker Compose lance une instance en 2 secondes

Les 4 tables et pourquoi

Table asn

```
CREATE TABLE asn (
  id SERIAL PRIMARY KEY,
  asn_number INTEGER UNIQUE NOT NULL,
  name VARCHAR(255),
  country_code CHAR(2) NOT NULL,
  is_manrs_member BOOLEAN DEFAULT FALSE,
  manrs_score SMALLINT DEFAULT 0 CHECK (manrs_score BETWEEN 0 AND 4),
  action_filtering BOOLEAN DEFAULT FALSE,
  action_antispoofing BOOLEAN DEFAULT FALSE,
  action_coordination BOOLEAN DEFAULT FALSE,
  action_validation BOOLEAN DEFAULT FALSE,
  last_updated TIMESTAMP DEFAULT NOW()
);
```

Pourquoi cette structure ?

- asn_number : c’est l’identifiant unique du monde réel (AS28683, AS37090, etc.)
- name : le nom de l’opérateur (exemple : “SBIN Bénin”) pour afficher au lieu du numéro
- country_code : pour le filtre “affiche les ASN du Bénin” dans la page pays

- `is_manrs_member` : booléen simple, vient de l'API MANRS v2
- `manrs_score` : somme des 4 actions (0 à 4). CHECK garantit que c'est entre 0 et 4, sinon INSERT échoue
- 4 colonnes `action_*` : booléens. True si cette action MANRS est complétée
- `last_updated` : timestamp pour savoir si les données sont fraîches (collectée le 24 juin à 4h41 ?)

Clé étrangère vers quoi ? Aucune. asn est au sommet de la hiérarchie. Les prefixes pointent vers lui.

Table prefixes

```
CREATE TABLE prefixes (
  id SERIAL PRIMARY KEY,
  asn_id INTEGER REFERENCES asn(id) ON DELETE CASCADE,
  prefix VARCHAR(50) NOT NULL,
  roa_status VARCHAR(20) CHECK (roa_status IN ('valid', 'invalid', 'not-found')),
  roa_asn INTEGER,
  last_checked TIMESTAMP DEFAULT NOW()
);
```

Pourquoi cette structure ?

- Un ASN peut annoncer plusieurs préfixes (exemple : AS28683 en annonce 54)
- Chaque préfixe a un statut ROA indépendant
- `roa_status` est un ENUM en pratique (CHECK le force)
- `roa_asn` : si le ROA est valide, quel ASN le signe ? (Permet la détection de hijacking : si `roa_asn` $\neq$ `asn_number` et statut = invalid $\rightarrow$ hijacking probable)
- ON DELETE CASCADE : si tu supprimes un ASN, ses préfixes disparaissent aussi (pas de données orphelines)

Pourquoi ne pas garder juste le % de couverture ROA dans la table asn ?

Parce que le % change chaque collecte (des préfixes sont ajoutés/supprimés). Il faut recalculer à chaque fois. Si tu stockes le %, tu dois aussi stocker la liste complète des préfixes pour le recalculer. Donc tu stockes la liste complète dans `prefixes` et tu calcules le % à la volée avec une LATERAL JOIN.

Table countries

```
CREATE TABLE countries (
  id SERIAL PRIMARY KEY,
  country_code CHAR(2) PRIMARY KEY,
  country_name VARCHAR(100) NOT NULL,
  total_asn INTEGER DEFAULT 0,
  manrs_members INTEGER DEFAULT 0,
  avg_manrs_score DECIMAL(3,2) DEFAULT 0,
  roa_coverage_pct DECIMAL(5,2) DEFAULT 0,
  last_updated TIMESTAMP DEFAULT NOW()
);
```

Pourquoi cette table ?

- Agrégats pré-calculés pour la page dashboard (les 16 pays avec leurs stats)
- Au lieu d'interroger les 477 ASN à chaque fois, tu fais une simple query sur 16 pays
- Ces stats sont **recalculées à chaque collecte** par une fonction SQL (UPDATE ... SET ... FROM (SELECT ...))

Formule de `roa_coverage_pct` pour un pays :

`count(préfixes avec roa_status='valid') / count(tous les préfixes) * 100`

Pour tous les ASN du pays.

Table ai_recommendations (réservée)

```
CREATE TABLE ai_recommendations (
  id SERIAL PRIMARY KEY,
  asn_id INTEGER REFERENCES asn(id) ON DELETE CASCADE,
  language VARCHAR(10) DEFAULT 'fr',
  content TEXT,
  generated_at TIMESTAMP DEFAULT NOW()
);
```

Pourquoi ? Cette table n'est pas utilisée dans la v1 (module IA exclu). Elle est là pour la v2 si tu décides d'intégrer un module IA qui génère automatiquement des recommandations personnalisées par ASN.

Le script schema.sql — Ordre d'exécution

```
-- 1. Créer les tables
CREATE TABLE asn (...);
CREATE TABLE prefixes (...);
CREATE TABLE countries (...);
CREATE TABLE ai_recommendations (...);

-- 2. Insérer les 16 pays de référence
INSERT INTO countries (country_code, country_name) VALUES
  ('BJ', 'Bénin'),
  ('BF', 'Burkina Faso'),
  ...
ON CONFLICT DO NOTHING; -- Si on relance le script, pas d'erreur

-- 3. Créer les indexes pour accélérer les requêtes
CREATE INDEX idx_asn_country ON asn(country_code);
CREATE INDEX idx_prefixes_asn ON prefixes(asn_id);
CREATE INDEX idx_asn_number ON asn(asn_number);
```

Pourquoi les indexes ?

- Query “affiche tous les ASN du Bénin” → cherche dans asn WHERE country_code='BJ' → sans index, ça scan 477 lignes. Avec index, ça va droit à la cible
- Query “affiche les préfixes d'un ASN” → cherche dans prefixes WHERE asn_id=123 → pareil

Qu'est-ce qui se passe au démarrage ?

1. Docker Compose lance PostgreSQL
2. PostgreSQL crée une base vide
3. PostgreSQL exécute /docker-entrypoint-initdb.d/schema.sql (monté depuis le host)
4. Les 4 tables et les 16 pays existent
5. Les collecteur et API peuvent se connecter

Une seule fois : Si tu relances docker compose up sans docker compose down -v (qui supprime les volumes), la base existe déjà, et le script ne s'exécute pas (ou refuse de recréer les tables).

Chapitre 3 — Module 2 : Le collecteur**Pourquoi un collecteur Python ?**

-  Facile à orchestrer : boucle for country in 16_countries: ...
-  Requêtes HTTP : librairie requests fait ça très simplement
-  Rate limiting : time.sleep(0.5) entre chaque appel
-  Try/except : si un ASN échoue, tu continues avec le suivant

-  Logs : tu sais ce qui s'est passé (277 ASN collectés, 3 erreurs)
-  Docker-compatible : l'image de backend peut lancer le collecteur ou l'API selon la commande

Architecture du collecteur

```
backend/collector/
├── __init__.py      (package vide)
├── config.py        (16 pays, URLs des APIs, clé MANRS)
├── apis.py          (fonctions d'appel aux 5 sources)
├── db.py            (fonctions de persistance PostgreSQL)
└── main.py          (orchestration + scheduling)
```

Fichier 1 : config.py

```
WEST_AFRICA_COUNTRIES = {
    "BJ": "Bénin",
    "BF": "Burkina Faso",
    ...
}

MANRS_API_BASE = "https://observatory.manrs.org/api/v2"
RIPE_STAT_BASE = "https://stat.ripe.net/data"
RPKI_VALIDATOR_BASE = "https://rpki-validator.ripe.net/api/v1"

API_DELAY_SECONDS = 0.5
COLLECTION_INTERVAL_HOURS = 6
```

Centraliser les constantes : si une URL change, tu la modifies une fois. C'est un pattern dans tous les projets.

Fichier 2 : apis.py — Les 5 fonctions

Fonction 1 : fetch_manrs_asns_for_country(code)

```
def fetch_manrs_asns_for_country(country_code: str) -> list[dict]:
    """
    MANRS v2 /ases?economies=XX
    Retourne : [{"id": 28683, "name": "SBIN", ...}, ...]
    """
```

Flux : 1. Interroge <https://observatory.manrs.org/api/v2/ases?economies=BJ> 2. MANRS retourne une page (pagination possible) 3. Si nextPageToken existe, refait une requête avec le token 4. Accumule tous les ASN de toutes les pages dans results 5. Retourne la liste

Exemple réel : - Bénin → 16 ASN - Nigeria → 224 ASN - Mali → 5 ASN

Pourquoi pagination ? MANRS retourne 100 ASN par page. Nigeria en a 224 → besoin de 3 pages.

Fonction 2 : fetch_manrs_participants()

```
def fetch_manrs_participants() -> set[int]:
    """
    MANRS v2 /participants (Auth: Bearer <KEY>)
    Retourne : {28683, 37282, 37424, ...} (set de ASN)
    """
```

Flux : 1. Header: Authorization: Bearer <MANRS_API_KEY> 2. Interroge <https://observatory.manrs.org/api/v2/participants> 3. Pour chaque participant, extrait les ASN qu'il gère 4. Accumule dans un set (pas de doublons) 5. Retourne le set

Résultat réel : 1813 membres MANRS dans le monde (au 24 juin 2026)

Pourquoi un set ? Tu vas faire `if asn_number in manrs_members:` → le test d'appartenance dans un set est $O(1)$, pas $O(n)$.

Pourquoi une fois, pas par pays ? Parce que tu l'utilises pour tous les 16 pays. Tu l'appelles une fois au démarrage, tu le réutilises.

Fonction 3 : `fetch_announced_prefixes(asn_number)`

```
def fetch_announced_prefixes(asn_number: int) -> list[str]:
    """
    RIPE Stat announced-prefixes
    Retourne : ["137.255.1.0/24", "196.46.153.0/24", ...]
    """
```

Flux : 1. Interroge RIPE Stat `/data/announced-prefixes/data.json?resource=AS28683` 2. RIPE retourne une liste de préfixes 3. Extrait les strings des préfixes 4. Retourne la liste

Exemple : AS28683 → 54 préfixes

Pourquoi RIPE Stat et pas MANRS ? MANRS ne fournit pas la liste des préfixes. RIPE Stat est la source de vérité pour le BGP global.

Fonction 4 : `validate_roa(asn_number, prefix)`

```
def validate_roa(asn_number: int, prefix: str) -> dict:
    """
    RIPE NCC RPKI Validator
    Retourne : {
        "prefix": "137.255.1.0/24",
        "roa_status": "valid" | "invalid" | "not-found",
        "roa_asn": 28683
    }
    """
```

Flux : 1. Interroge RIPE Validator `/api/v1/validity/<asn>/<prefix>` - Exemple :

`/api/v1/validity/28683/137.255.1.0%2F24` 2. RIPE retourne : `json { "validated_route": { "validity": { "state": "valid", "VRPs": { "matched": [{"asn": "AS28683", ...}] } } } }` 3. Extrait state et asn signataire 4. Retourne le dict

Les 3 statuts : - valid : ROA existe et correspond (sûr) - invalid : ROA existe mais ne correspond pas (dangereux, possible hijacking) - not-found : aucun ROA (non protégé, mais pas nécessairement compromis)

Pourquoi c'est lent ? Tu dois faire 1 appel par préfixe. $6810 \text{ préfixes} \times 0.5s \text{ rate limit} = 55 \text{ minutes}$. C'est la cause de la durée totale de collecte (111 min).

Fonction 5 : `fetch_peeringdb_info(asn_number)`

```
def fetch_peeringdb_info(asn_number: int) -> dict:
    """
    PeeringDB API
    Retourne : {
        "on_peeringdb": True,      # pour coordination
        "has_irr": True,          # pour filtering
        "has_policy": True        # bonus, non utilisé
    }
    """
```

Flux : 1. Interroge <https://www.peeringdb.com/api/net?asn=28683> 2. PeeringDB retourne une liste de réseaux 3. Prend le premier (il y en a généralement 1) 4. Extrait irr_as_set (non-vidé ?) et état membre PeeringDB 5. Retourne le dict

Logique : - on_peeringdb = True → l'opérateur est connu et maintient des contacts publics → bonne coordination - has_irr = True → l'opérateur publie son IRR as-set → capable de filtering

Bonus check_antispoofing(asn) (scraping CAIDA) :

```
def _load_spoof_data() -> dict[int, float]:
    """
    Scrape https://spoofing.caida.org/as_stats.php
    Parse le HTML avec regex pour extraire ASN et % blocking
    Retourne : {28683: 0.0, 37282: 87.5, ...}
    """
```

Pourquoi scraping ? CAIDA n'a pas d'API. Il faut parser la page HTML. **Danger :** si CAIDA change le format, ça casse.

Cache : _spoof_data global. Tu le charges une fois, tu le réutilises pour tous les ASN.

Fichier 3 : db.py — Persistance

Fonction 1 : upsert_asn(conn, asn_data)

```
INSERT INTO asn (...) VALUES (...)
ON CONFLICT (asn_number) DO UPDATE SET ...
```

Pourquoi UPSERT ? À chaque collecte, tu réinsères les mêmes ASN (477). Sans UPSERT, tu aurais des PRIMARY KEY violations. Avec UPSERT, tu mets à jour les valeurs existantes.

Effet : Ton collecteur peut tourner à l'infini. Si tu le relances, ça met à jour les données sans créer de doublons.

Fonction 2 : upsert_prefixes(conn, asn_id, prefixes_list)

```
DELETE FROM prefixes WHERE asn_id = ?
INSERT INTO prefixes (...) VALUES (batch)
```

Pourquoi DELETE puis INSERT ? Parce que les préfixes changent : - Un ASN qui avait 54 préfixes peut en avoir 53 la collecte suivante - Un préfixe peut passer de valid à not-found si le ROA expire

Au lieu de tracker les changements, tu nettoies et réinsères. C'est plus simple et plus fiable.

Fonction 3 : update_country_stats(conn, country_code)

```
UPDATE countries SET
    total_asn = (SELECT COUNT(*) FROM asn WHERE country_code = ?),
    manrs_members = (SELECT COUNT(*) FILTER (WHERE is_manrs_member) FROM asn WHERE ...),
    avg_manrs_score = (SELECT AVG(manrs_score) FROM asn WHERE ...),
    roa_coverage_pct = (SELECT 100*COUNT(*) FILTER (WHERE roa_status='valid') / COUNT(*)
                        FROM prefixes JOIN asn ...),
    last_updated = NOW()
```

Formule clé pour roa_coverage_pct :

```
COALESCE(
    100.0 * COUNT(*) FILTER (WHERE p.roa_valid > 0)
    / NULLIF(COUNT(*) FILTER (WHERE p.total_prefixes > 0), 0),
```

```
0
) AS roa_pct
```

Traduction : - Compte les préfixes avec roa_status = 'valid' - Divise par le nombre total de préfixes - Multiplie par 100 pour avoir un pourcentage - Si aucun préfixe (edge case), retourne 0

Fichier 4 : main.py — Orchestration

Fonction collect_asns_for_country(country_code, country_name, manrs_members)

Pour chaque ASN du pays :

1. fetch_announced_prefixes() → liste de préfixes
 2. Pour chaque préfixe :
 - validate_roa() → statut
 3. fetch_peeringdb_info() → coordination + filtering
 4. check_antispoofing() → anti-spoofing
 5. Calculer les 4 booléens
 6. upsert_asn() → sauvegarde en base
 7. upsert_prefixes() → sauvegarde préfixes en base
 8. update_country_stats() → recalcule les agrégats pays
- Log : AS28683 – SBIN – membre:False – 54 préfixes – ROA:92.59%

Fonction run_full_collection()

1. fetch_manrs_participants() → charge les 1813 membres
2. Pour chaque pays (16) :
 - collect_asns_for_country() → traite tous les ASN
3. Log final : Collecte terminée en 111.4 minutes

Fonction main()

1. run_full_collection() → une fois au démarrage
2. schedule.every(6).hours.do(run_full_collection) → relance toutes les 6h
3. Boucle infinie : schedule.run_pending() + sleep(60)

Pourquoi scheduling ? Tu veux que les données se mettent à jour automatiquement. Sans scheduling, le collecteur s'arrête après une collecte.

Points sensibles du collecteur

1. Rate limiting (0.5s entre les requêtes)

Si tu envoies 6810 requêtes sans délai, les APIs te bloquent (429 Too Many Requests). La fonction _get() inclut time.sleep(0.5) à chaque appel.

2. try/except par ASN

```
for asn in asn_list:
    try:
        # traitement
    except Exception:
        logger.exception("Erreur sur AS%d", asn_number)
        continue # → continue avec l'ASN suivant
```

Si AS12345 timeout, tu ne veux pas que ça casse toute la collecte.

3. CAIDA Spoofer = cache global

Première fois que tu appelles check_antispoofing(asn), il scrape la page HTML complète et charge 1442 ASN en mémoire. Ensuite, tu cherches dans le dict. Sans le cache, tu scrapais 477 fois.

4. Logs à chaque étape

```

2026-06-24 04:41:16 [INFO] === Début de la collecte complète ===
2026-06-24 04:41:19 [INFO] 1813 ASN membres MANRS chargés
2026-06-24 04:41:20 [INFO] Collecte Bénin (BJ)...
2026-06-24 04:41:21 [INFO] 16 ASN trouvés pour BJ
2026-06-24 04:41:50 [INFO] AS28683 (SBIN) – membre:False – 54 préfixes – ROA:92.59%
...
2026-06-24 06:29:19 [INFO] === Collecte terminée en 111.4 minutes ===

```

Ces logs te permettent de déboguer si quelque chose échoue.

Chapitre 4 — Module 3 : API Backend

Pourquoi FastAPI ?

-  Async : mieux que Flask pour I/O (requêtes DB)
-  Validation automatique : FastAPI valide les paramètres et retourne des erreurs claires
-  Docs auto : /docs génère une doc Swagger interactive
-  Performance : pour 477 ASN × 5 pages = 2385 requêtes/jour, FastAPI c'est du chiffre léger

Les 5 endpoints

GET /api/stats

Que retourne ?

```

{
  "total_asn": 477,
  "manrs_members": 11,
  "avg_manrs_score": 0.87,
  "roa_coverage_pct": 51.38
}

```

SQL :

SELECT

```

COUNT(*) AS total_asn,
COUNT(*) FILTER (WHERE is_manrs_member) AS manrs_members,
ROUND(AVG(manrs_score)::numeric, 2) AS avg_manrs_score,
COALESCE(ROUND(
  100.0 * SUM(p.valid) / NULLIF(SUM(p.total), 0), 2
), 0) AS roa_coverage_pct
FROM asn a
LEFT JOIN LATERAL (
  SELECT
    COUNT(*) AS total,
    COUNT(*) FILTER (WHERE roa_status = 'valid') AS valid
  FROM prefixes WHERE asn_id = a.id
) p ON TRUE

```

Ce que ça fait : - Pour chaque ASN, calcule ses stats ROA (via LATERAL JOIN) - Agrège tout à niveau global

Utilisation : Page d'accueil : affiche les 4 stat cards (477 ASN, 11 MANRS, etc.)

GET /api/countries

Que retourne ?

```
[
  {
    "country_code": "BJ",
    "country_name": "Bénin",
    "total_asn": 16,
    "manrs_members": 0,
    "avg_manrs_score": 0.69,
    "roa_coverage_pct": 12.5,
    "last_updated": "2026-06-24T04:41:16"
  },
  ...
]
```

SQL : Simple, c'est la table countries pré-calculée.

Utilisation : Tableau du dashboard : classement des 16 pays.

GET /api/countries/{code}

Que retourne ?

```
{
  "country_code": "BJ",
  "country_name": "Bénin",
  "total_asn": 16,
  "manrs_members": 0,
  "avg_manrs_score": 0.69,
  "roa_coverage_pct": 12.5,
  "last_updated": "2026-06-24T04:41:16",
  "asns": [
    {
      "asn_number": 28683,
      "name": "SBIN",
      "is_manrs_member": false,
      "manrs_score": 2,
      "action_filtering": false,
      "action_antispoofing": false,
      "action_coordination": true,
      "action_validation": true,
      "roa_coverage_pct": 92.59
    },
    ...
  ]
}
```

SQL : Stats du pays + liste de ses ASN avec couverture ROA calculée.

Utilisation : Page pays (ex : /country/BJ).

GET /api/asn/{number}

Que retourne ?

```
{
  "asn_number": 28683,
  "name": "SBIN",
  "country_code": "BJ",
  "is_manrs_member": false,
  "manrs_score": 2,
  "actions": {
```

```

    "filtering": false,
    "anti_spoofing": false,
    "coordination": true,
    "validation": true
  },
  "prefixes": [
    {
      "prefix": "137.255.1.0/24",
      "roa_status": "valid"
    },
    {
      "prefix": "196.46.153.0/24",
      "roa_status": "not-found"
    }
  ],
  "roa_coverage_pct": 92.59,
  "last_updated": "2026-06-24T04:41:16"
}

```

SQL : Fiche complète ASN + ses préfixes.

Utilisation : Fiche ASN détaillée (ex : /asn/28683).

GET /api/search?q=query

Que retourne ?

Depend du query : - Si numeric (ex : ?q=28683) → exact match ASN - Sinon (ex : ?q=MTN) → LIKE search sur le nom (20 résultats max)

```

[
  {
    "asn_number": 37282,
    "name": "Main One Cable Company Nigeria Limited",
    "country_code": "NG"
  },
  ...
]

```

Utilisation : Barre de recherche du header.

Point sensible : CORS

```

app.add_middleware(
  CORSMiddleware,
  allow_origins=["*"],
  allow_methods=["*"],
  allow_headers=["*"],
)

```

Pourquoi ? Le frontend tourne sur http://localhost:3000, l'API sur http://localhost:8000. Sans CORS, le navigateur refuse les requêtes cross-origin.

En production ? Restreins à l'URL du frontend :

```
allow_origins=["https://observatoire.manrs.org"],
```

Point sensible : RealDictCursor

```

with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
    cur.execute(sql)

```

```
return [dict(row) for row in cur.fetchall()]
```

Pourquoi ? RealDictCursor retourne les lignes comme des dicts {"column": value} au lieu de tuples. Ça permet à FastAPI de sérialiser directement en JSON.

Chapitre 5 — Module 4 : Frontend React

Pourquoi React + TypeScript ?

- ✓ Composants réutilisables : une stat card, une table, une ligne de tableau
- ✓ État réactif : quand les données changent, l'interface se met à jour
- ✓ TypeScript : tu sais que `asn.name` est une string, pas un number. Les erreurs apparaissent à la compilation, pas en production
- ✓ Écosystème : Recharts pour graphiques, Leaflet pour carte, shadcn/ui pour composants

Architecture Atomic Design

```
Pages (5)
  ↓
Organisms (5 composants complexes)
  ↓
Molécules (5 combinaisons)
  ↓
Atoms (4 unités)
  ↓
UI (16 composants shadcn)
```

Exemple : page pays

```
<CountryPage>
├─ <StatValue> (stat card)      ← Atom
├─ <CountriesChart>            ← Organism
│   └─ <Recharts BarChart>    ← Librairie externe
├─ <AsnTable>                  ← Organism
│   └─ <Badge> (shadcn)       ← UI
```

Avantage : Si tu veux changer le style des stat cards, tu modifies le fichier `atoms/stat-value.tsx`, c'est automatiquement appliqué partout.

Hook use-fetch.ts

```
const { data, loading, error } = useFetch(() => api.getCountry("BJ"))
```

Flux : 1. Appel `api.getCountry("BJ")` au montage du composant 2. `loading = true` 3. Awaiting la réponse 4. `loading = false, data = {...}` 5. Composant se rerender avec data

Avantage : Réutilisable par toutes les pages. Pas besoin de dupliquer la logique fetch 5 fois.

Client API (lib/api.ts)

```
const API_BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:8000'

export const api = {
  getStats: () => fetch(`${API_BASE_URL}/api/stats`).then(r => r.json()),
  getCountries: () => fetch(`${API_BASE_URL}/api/countries`).then(r => r.json()),
  ...
}
```

Variable d'environnement : VITE_API_URL est remplacée à la compilation. En dev, c'est localhost:8000. En prod, c'est l'URL publique.

Couleurs ROA (lib/countries.ts)

```
export function roaColorHex(pct: number): string {
  if (pct > 60) return "#22c55e" // Vert
  if (pct > 25) return "#f59e0b" // Orange
  return "#ef4444" // Rouge
}
```

Utilisé par la carte (couleur des cercles) et les graphiques (couleur des barres).

Les 5 pages

Page	Route	Rôle
HomePage	/	Dashboard : stats + carte + chart + tableau
CountryPage	/country/:code	Détail pays : stats + chart ASN + tableau ASN
AsnDetailPage	/asn/:number	Fiche ASN : scorecard + pie ROA + tableau préfixes
SearchPage	/search?q=...	Résultats de recherche
AboutPage	/about	Explication MANRS/RPKI + sources

Point sensible : z-index Leaflet

```
.leaflet-container {
  z-index: 0 !important;
}
```

Sans ça, la carte passe au-dessus du header sticky au scroll. Avec ça, elle reste derrière.

Point sensible : Dark theme Leaflet

```
.leaflet-tile-pane {
  filter: brightness(0.6) invert(1) contrast(3) hue-rotate(200deg) saturate(0.3)
  brightness(0.7);
}
```

OpenStreetMap (les tuiles par défaut) est en mode clair. Ce filtre inverse les couleurs pour un rendu dark.

Chapitre 6 — Module 5 : Déploiement

docker-compose.yml — Services

Service	Image	Port	Init
db	postgres:15	5432	schema.sql
api	Dockerfile (backend)	8000	DATABASE_URL
collector	Dockerfile (backend, commande différente)	—	DATABASE_URL
frontend	Dockerfile (frontend)	3000	VITE_API_URL

Dockerfile backend (backend/Dockerfile)

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Le CMD par défaut lance l'API. Pour le collecteur, le docker-compose.yml override avec :

```
collector:
  command: python -m collector.main
```

Dockerfile frontend (à créer)

```
# Stage 1 : Build
FROM node:18-alpine AS builder
WORKDIR /app
COPY package.json package-lock.json .
RUN npm install
COPY . .
RUN npm run build

# Stage 2 : Serve
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 3000
CMD ["nginx", "-g", "daemon off;"]
```

Stage 1 : Build Vite → génère /dist

Stage 2 : Copie /dist dans nginx → sert du HTML/CSS/JS statique

Variables d'environnement

Variable	Où	Valeur
DATABASE_URL	collecteur + API	postgresql://manrs:changeme@db:5432/manrs_observatory
MANRS_API_KEY	collecteur	Clé obtenue sur MANRS Observatory
VITE_API_URL	frontend (build)	http://localhost:8000 (dev), https://api.observatoire.manrs.org (prod)

Healthcheck

```
db:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U manrs"]
    interval: 5s
    timeout: 3s
    retries: 10
```

Les services api et collector attendent que db soit healthy avant de démarrer. Sinon : - api tente de se connecter à PostgreSQL qui ne répond pas → crash - collector même chose → crash

Avec le healthcheck, Docker Compose attend jusqu'à 50 secondes que PostgreSQL soit prêt.

Chapitre 7 — Flux complet de données

[Utilisateur ouvre http://localhost:3000]



[Frontend React charge (build Vite)]



[HomePage monte]

```

├─ useFetch(() => api.getStats())
│   └─ GET http://localhost:8000/api/stats
│       └─ PostgreSQL query (1 ms)
│           └─ {total_asn: 477, ...}
├─ useFetch(() => api.getCountries())
│   └─ GET http://localhost:8000/api/countries
│       └─ SELECT * FROM countries (16 rows)
│           └─ [{country_code: "BJ", ...}, ...]
└─ Rendu des 4 stat cards + tableau 16 pays + carte
    └─ Utilisateur clique sur "Bénin"
        └─ Navigate to /country/BJ
            └─ CountryPage monte
                └─ useFetch(() => api.getCountry("BJ"))
                    └─ GET http://localhost:8000/api/countries/BJ
                        └─ SELECT * FROM countries + liste des ASN
                            └─ {country_code: "BJ", asns: [{...}, ...]}
                                └─ Affiche stats Bénin + graphique top 15 opérateurs + tableau 16

```

opérateurs

```

└─ Utilisateur clique sur AS28683
    └─ Navigate to /asn/28683
        └─ AsnDetailPage monte
            └─ useFetch(() => api.getAsn(28683))
                └─ GET http://localhost:8000/api/asn/28683
                    └─ SELECT asn + prefixes WHERE asn_id=...
                        └─ {asn_number: 28683, actions: {...},

```

prefixes: [{...}, ...]}

└─ Affiche scorecard 2/4 + pie ROA 92.59% +

tableau 54 préfixes

Chaque appel API retourne de la donnée JSON, React la reçoit, la stocke dans l'état, et se rerender.

Chapitre 8 — Pièges courants et comment les éviter

Piège 1 : CORS bloquer tout

Symptôme : Frontend envoie une requête, le navigateur log "CORS error", la requête ne part pas.

Cause : Le frontend (port 3000) appelle l'API (port 8000), navigateur demande permission (CORS). Sans CORSMiddleware, FastAPI refuse.

Solution : Ajouter le middleware CORS.

Piège 2 : Clé API MANRS expire

Symptôme : Le collecteur log "Pas de clé API MANRS — statut membre non disponible". Tous les `is_manrs_member` sont False.

Cause : La clé a expiré (toutes les 3 jours par défaut).

Solution : Voir la procédure de renouvellement dans le `GUIDE_MODULES.md`.

Piège 3 : Base de données vide

Symptôme : Frontend charge, mais le tableau est vide.

Cause : Tu as lancé `docker compose up` mais le collecteur n'a pas tourné. La base a les tables mais pas de données.

Solution : Lancer le collecteur manuellement, ou attendre qu'il tourne automatiquement (c'est en background dans le conteneur).

Piège 4 : Frontend ne peut pas communiquer avec l'API

Symptôme : "Failed to fetch /api/stats" dans la console du navigateur.

Cause 1 : L'API n'est pas lancée (conteneur a crashé).

Cause 2 : `VITE_API_URL` n'est pas définie (frontend cherche sur un port inexistant).

Solution : `docker compose logs api` (vérifier qu'il tourne), ou export `VITE_API_URL=http://localhost:8000` avant `npm run dev`.

Piège 5 : Rate limiting tue la collecte

Symptôme : Collecteur tourne mais l'API log "429 Too Many Requests".

Cause : `API_DELAY_SECONDS = 0` (pas de délai). Les APIs bloquer après 50 requêtes/seconde.

Solution : Garder `API_DELAY_SECONDS = 0.5` (il est dans `config.py`).

Piège 6 : Statut ROA mélangé avec % ROA

Symptôme : Frontend affiche "Valid" comme un nombre (ex : "42%") au lieu d'un statut.

Cause : Confusion entre `prefixes[].roa_status` ("valid"/"invalid"/"not-found") et `asn.roa_coverage_pct` (51.4).

Solution : Lire l'API response attentivement. Dans `/api/asn/{number}`, `prefixes` contient les statuts, `roa_coverage_pct` est un nombre.

Chapitre 9 — Mesure de succès

Objectif principal réussi ?

Tu dois pouvoir répondre à la question :

Quel est l'état de la sécurité du routage Internet en Afrique de l'Ouest ?

Avec des chiffres à l'appui : - 477 ASN surveillés - 11 membres MANRS (2.3%) - 51.4% de couverture ROA - 0% d'anti-spoofing (aucun ASN testé bloque le spoofing) - Classement par pays (Mali 60%, Nigeria 28.6%, Mauritanie 0%)

Test simple end-to-end

1. `docker compose up`
2. Attendre que PostgreSQL soit healthy (50s max)
3. Attendre que le collecteur finisse une collecte (111 min)
4. Ouvrir `http://localhost:3000`
5. Cliquer sur Bénin → voir les 16 ASN
6. Cliquer sur AS28683 → voir 54 préfixes
7. Chercher "MTN" → voir les résultats

Si tout ça fonctionne, tu as réussi.

Conclusion

L'architecture est pensée pour être **modulaire** et **résiliente** :

- **Modulaire** : Chaque module (DB, collecteur, API, frontend) peut être développé indépendamment et testé isolément.
- **Résiliente** : Si un ASN échoue, le collecteur continue. Si l'API crash, tes données sont sauvegardées. Si tu relances le collecteur, il remet à jour sans doublon (UPSERT).

Le “pourquoi” derrière chaque fichier, fonction et décision, c'est :

Répondre à la question centrale avec des données agrégées de 5 sources publiques, sans coopération des opérateurs, de manière automatisée et visualisable.

C'est ça, l'observatoire MANRS.